

Loan Default Prediction

Jim Ryan

Originally completed March 2020; updated 2026

Executive Summary

A logistic regression model trained on ~28,000 loans achieved an **AUC of 0.713** on held-out test data, showing solid ability to rank borrowers by default risk.

A threshold analysis found that declining high-risk loans improved test-set profit only marginally (+1.4% at a 0.38 threshold), and that stricter thresholds sharply reduced profit because interest from good loans outweighed losses from defaults.

Recommendation: the model is better suited to risk-based pricing or credit limit setting than to approve/decline decisions.

Part 1: Introduction

```
knitr::opts_chunk$set(warning = FALSE, message = FALSE)
```

For this project I will use logistic regression to predict which applicants are likely to default on their loans. I am going to use the loans50k data set that was given to us as part of this assignment. This data set contains 50,000 loans in various statuses and various amounts. There are 32 available variables in the data set. My response variable will be the status variable. I will start by preparing and cleaning the data.

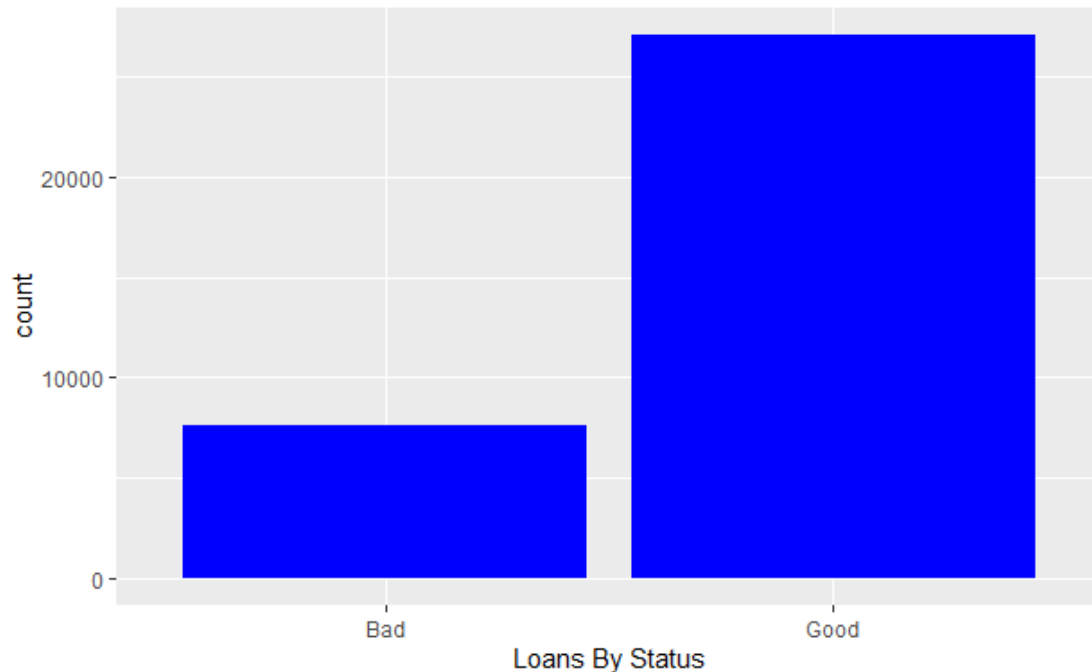
```
LoansDT <- fread("Loans50k.csv")
```

Part 2: Preparing and Cleaning the Data

The data contains different variables collected as either part of the application process or information from when the loan was disbursed. I am trying to predict which variable or variables will allow me to predict whether a loan becomes default or not. I removed all loans that did not have a status of 'Fully Paid', 'Default' or 'Charged Off'. Then I copied the response variable – status – into another field called status2. I then updated the status of loans that were Fully Paid to 'Good' and the ones that were 'Default' or 'Charged Off' to 'Bad'.

There are 27,074 loans or 78% with a status of 'Good' and 7,581 loans or 22 percent with a status of 'Bad'. Here is a Histogram of that shows the number of good and bad loans after the status has been updated.

```
#bar plot on status
p1 <- ggplot(data=LoansDT, aes(x=status2)) +
  geom_bar(fill="blue") +
  xlab("Loans By Status")
p1
```



```
#Update good to 1 bad to 0 and make var a factor
LoansDT <- sqldf(c("UPDATE LoansDT SET status2 = 1 where status2 = 'Good'", "UPDATE LoansDT SET status2 = 0 where status2 = 'Bad'", "Select * from LoansDT"))

# set status field as factor
LoansDT$status2 <- as.factor(LoansDT$status2)
```

I used supply with a sum on the NA in the data set to determine which variable had NA in them and how many there were. I found that the only 3 variables that had NA values in them were revolRatio (15 NA), bcOpen (360 NA) and bcRatio (384 NA). Since the missing values were only around 1 percent of the total number for these variables, I imputed them using the mean of all the values for that specific variable.

Then I focused my attention on the employment variable. There were 15,268 different values for employment including 1918 where the value is blank. I selected all rows where the value had a count greater than 100. I updated 1789 values of the employment variable that had the word manager in the to be just 'Manager'. There were 934 different variations of the word 'Teacher', so I updated them all to be teacher.

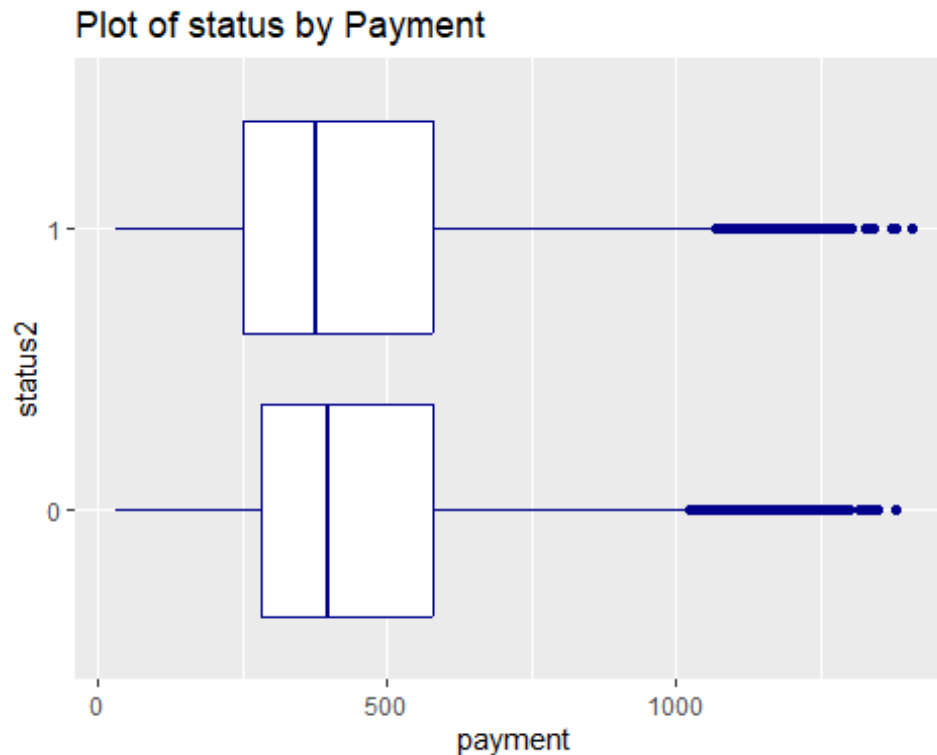
The 'Grade variable was updated from 'A' thru 'G' to 0 thru 6 and made into a factor. I update the 'verified' variable by updating those with a value of 'Source Verified' to 'Verified' and made that variable a factor. I did something similar with the home variable. I set the variable to OWN when the value was mortgage so then there were just 2 categorical variables - 'own' and 'rent' made the variable a factor. The variable term only had 2 values (36 months and 60 months) so the variable term2 was updated to a factor.

I removed the following variables because more than 25 percent of them were 0: pubRec, delinq2yr, and inq6mth. I also dropped the original status variable.

Part 4: Exploring and Transforming the Data

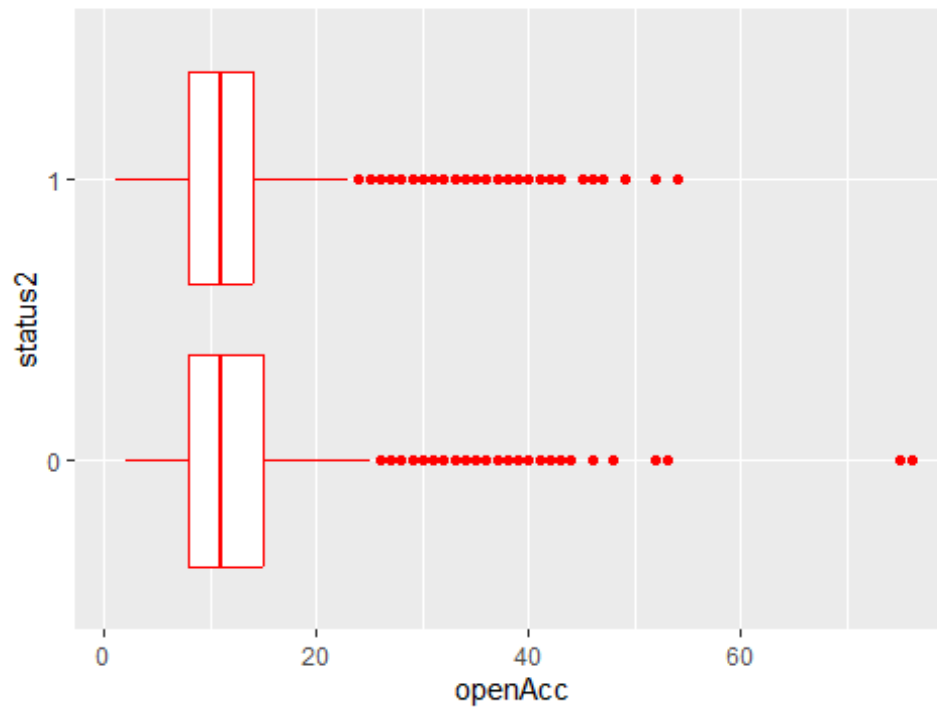
Now I delve into some data exploration and transformation. First I do some exploration on the data set as a whole. I create box plots of payment and status to see if there is an observable relationship between payment and status. There is no observable relationship between payment and status for the entire dataset. Then I did the same for openAcc income and loanID. The only one that showed an obvious relationship was totalBal. The 'LoanID' variable looks constant no matter what the status. I assume this is just a loan Identifier and will drop it. The other 3 variables I will keep for part 5.

```
par(mfrow=c(2,2))
ggplot(aes(x=payment,y=status2),data=LoansDT)+
  geom_boxplot(color='darkblue') + ggtitle("Plot of status by Payment")
```



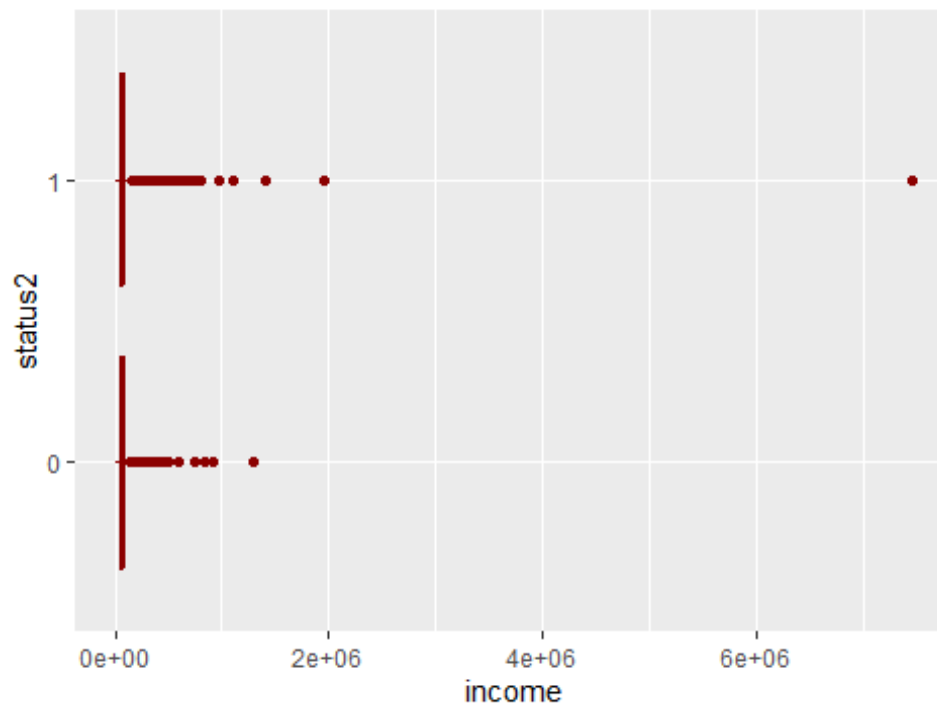
```
ggplot(aes(x=openAcc,y=status2),data=LoansDT)+
  geom_boxplot(color='red') + ggtitle("Plot of status by openAcc")
```

Plot of status by openAcc

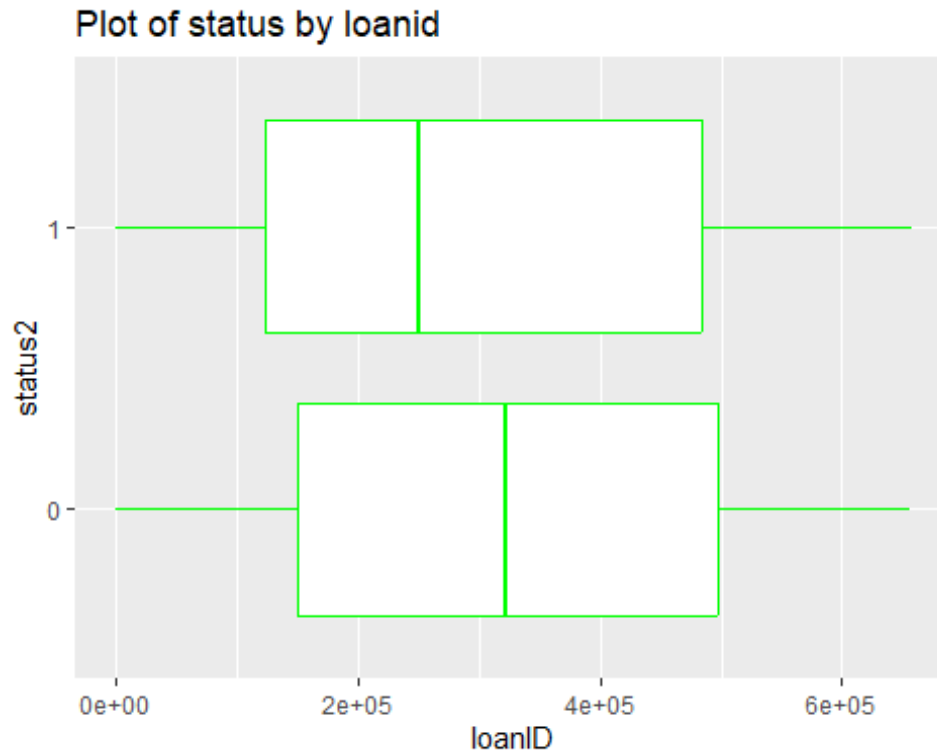


```
ggplot(aes(x=income,y=status2),data=LoansDT)+  
  geom_boxplot(color='darkred') + ggtitle("Plot of status by Income")
```

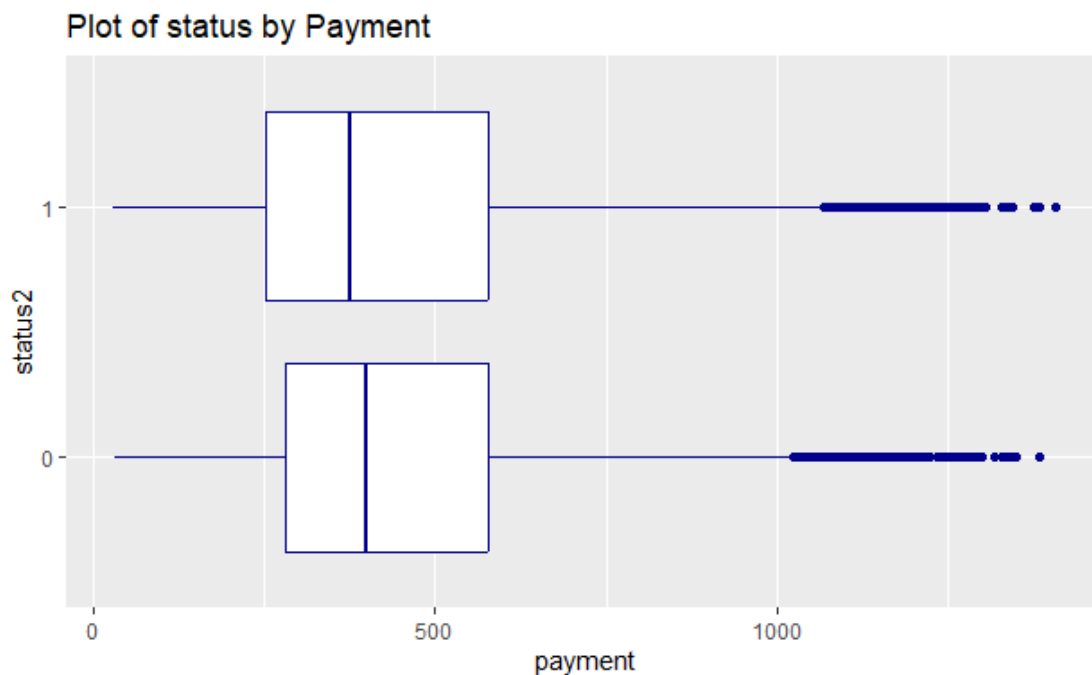
Plot of status by Income



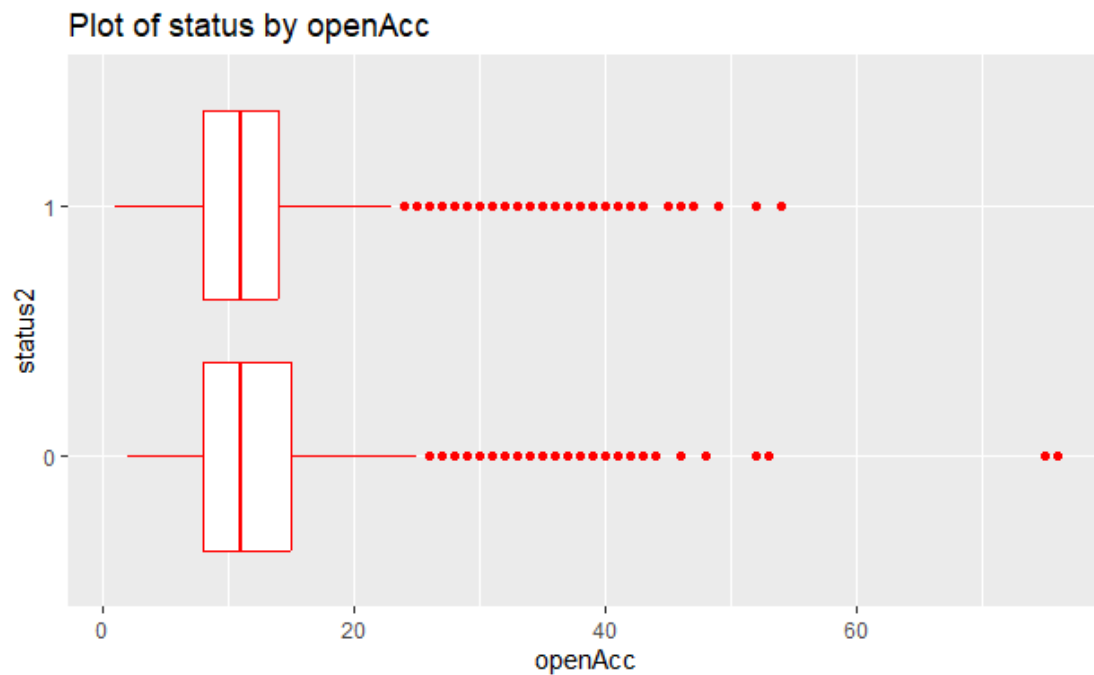
```
ggplot(aes(x=loanID,y=status2),data=LoansDT)+
  geom_boxplot(color='green') + ggtitle("Plot of status by loanid")
```



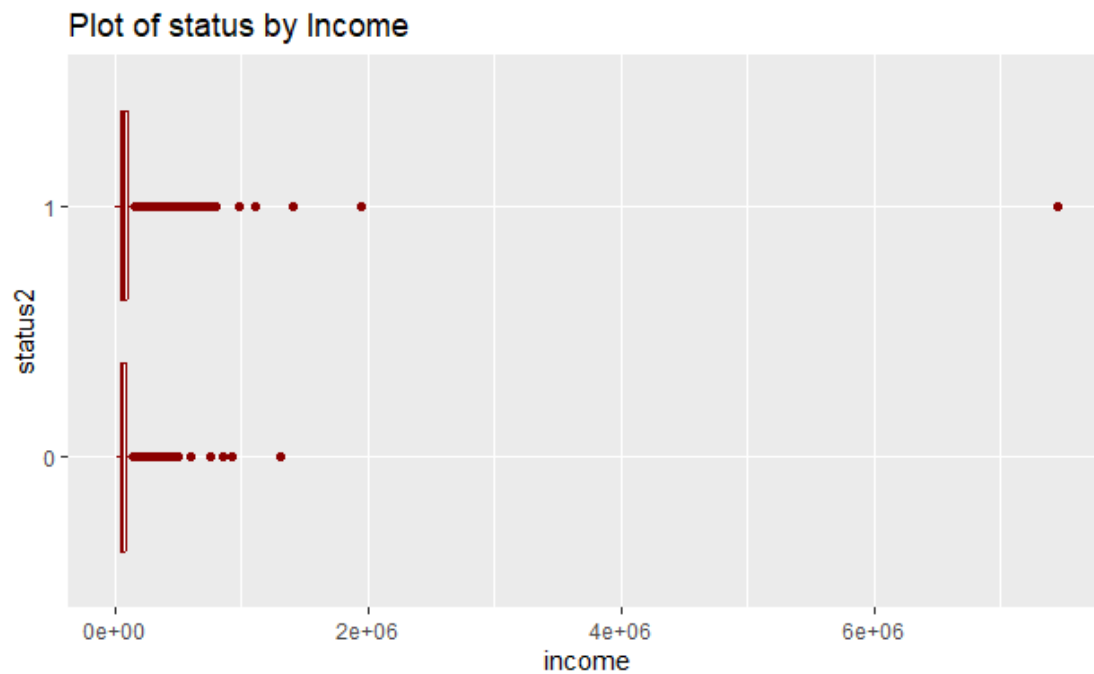
```
par(mfrow=c(2,2))
ggplot(aes(x=payment,y=status2),data=LoansDT)+
  geom_boxplot(color='darkblue') + ggtitle("Plot of status by Payment")
```



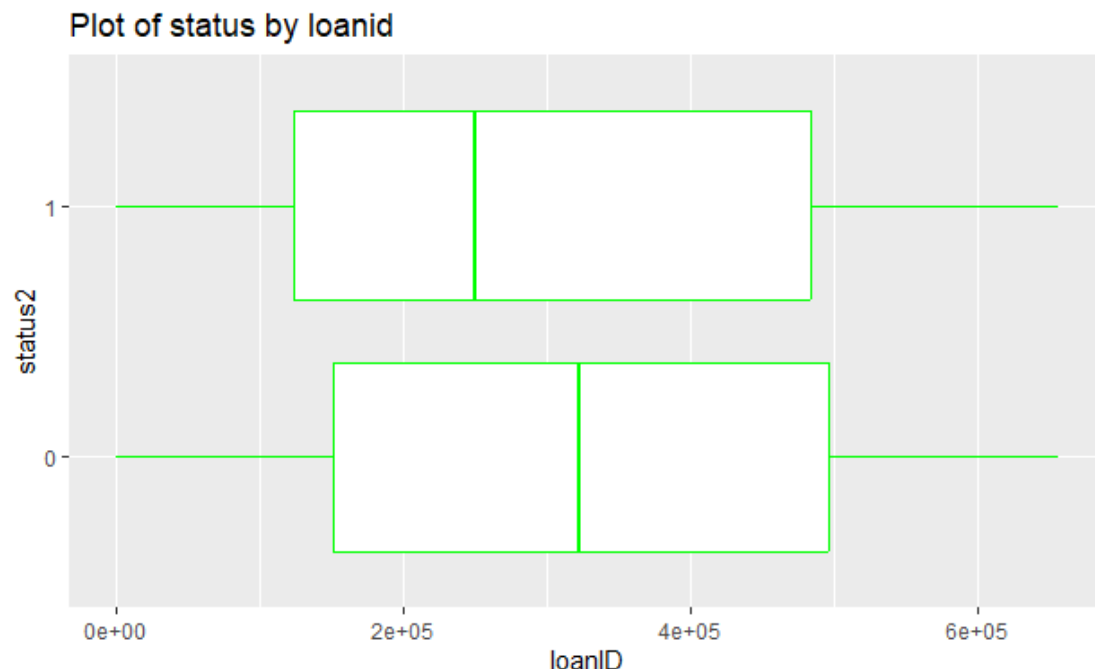
```
ggplot(aes(x=openAcc,y=status2),data=LoansDT)+
  geom_boxplot(color='red') + ggtitle("Plot of status by openAcc")
```



```
ggplot(aes(x=income,y=status2),data=LoansDT)+
  geom_boxplot(color='darkred') + ggtitle("Plot of status by Income")
```



```
ggplot(aes(x=loanID,y=status2),data=LoansDT)+
  geom_boxplot(color='green') + ggtitle("Plot of status by loanid")
```



Then I divided up my modified data frame into good and bad loans based on the status2 variable. Then I made bar graphs of some of the categorical variables divided by good and bad loans to see if there were any obvious relationships. I plotted the 'term', 'verified' and home variable against status2. For the term variable 'bad' loans have a greater percentage of loans with 60 month terms than the 'good' loans. It appears that there might be some relationship between the 'verified' and term variables and status. I will keep these variables for part 5.

```
library("gridExtra")
loans_bad <- sqldf("Select * from LoansDT where status2 = 0")
loans_good <- sqldf("Select * from LoansDT where status2 = 1")

p1 <- ggplot(loans_bad, aes(status2, fill = term)) + facet_wrap(~term) +
geom_bar()
p2 <- ggplot(loans_good, aes(status2, fill = term)) + facet_wrap(~term) +
geom_bar()

p3 <- ggplot(loans_bad, aes(status2, fill = verified)) +
facet_wrap(~verified) + geom_bar()
p4 <- ggplot(loans_good, aes(status2, fill = verified)) +
facet_wrap(~verified) + geom_bar()

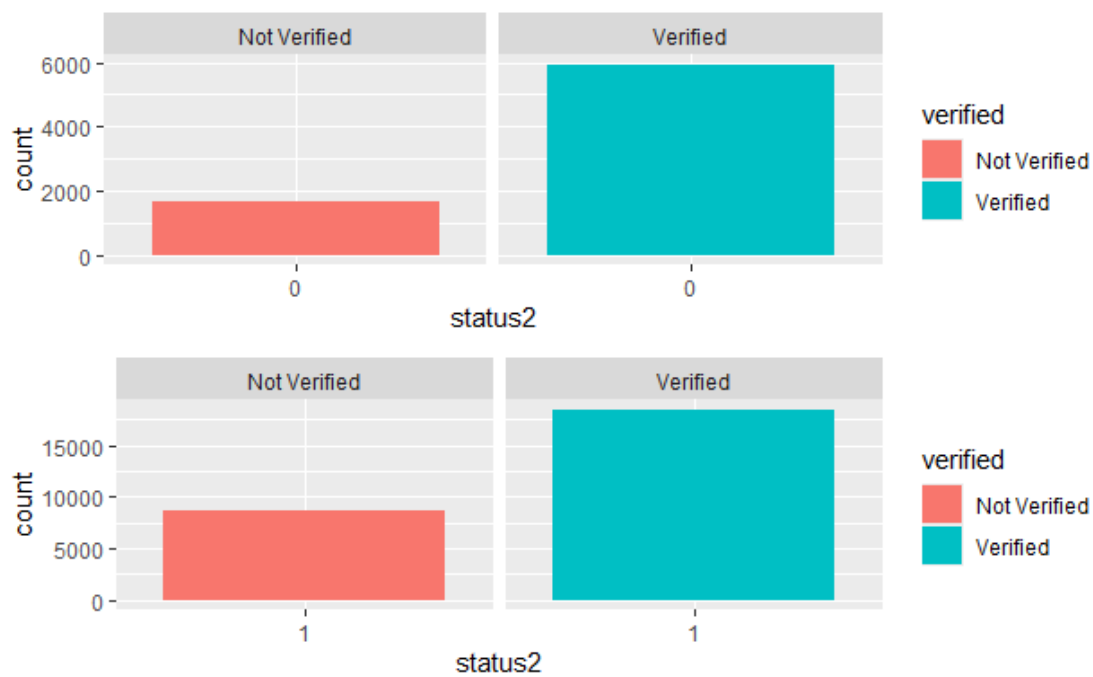
p5 <- ggplot(loans_bad, aes(status2, fill = home)) + facet_wrap(~home) +
geom_bar()
p6 <- ggplot(loans_good, aes(status2, fill = home)) + facet_wrap(~home) +
```

```
geom_bar()
```

```
grid.arrange(p1,p2)
```



```
grid.arrange(p3,p4)
```

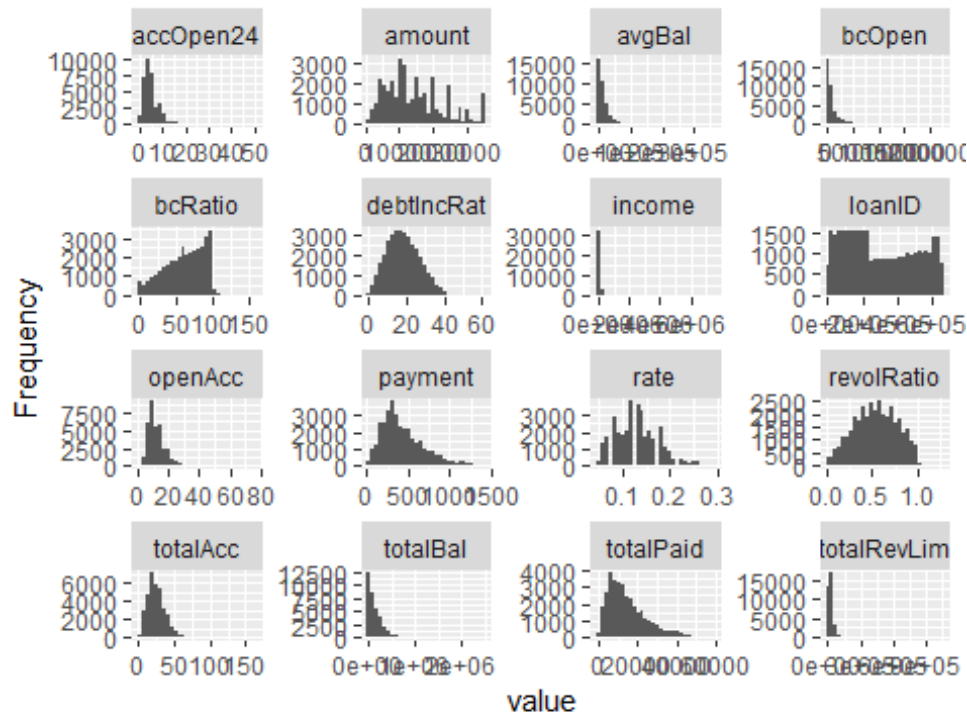


```
grid.arrange(p5,p6)
```

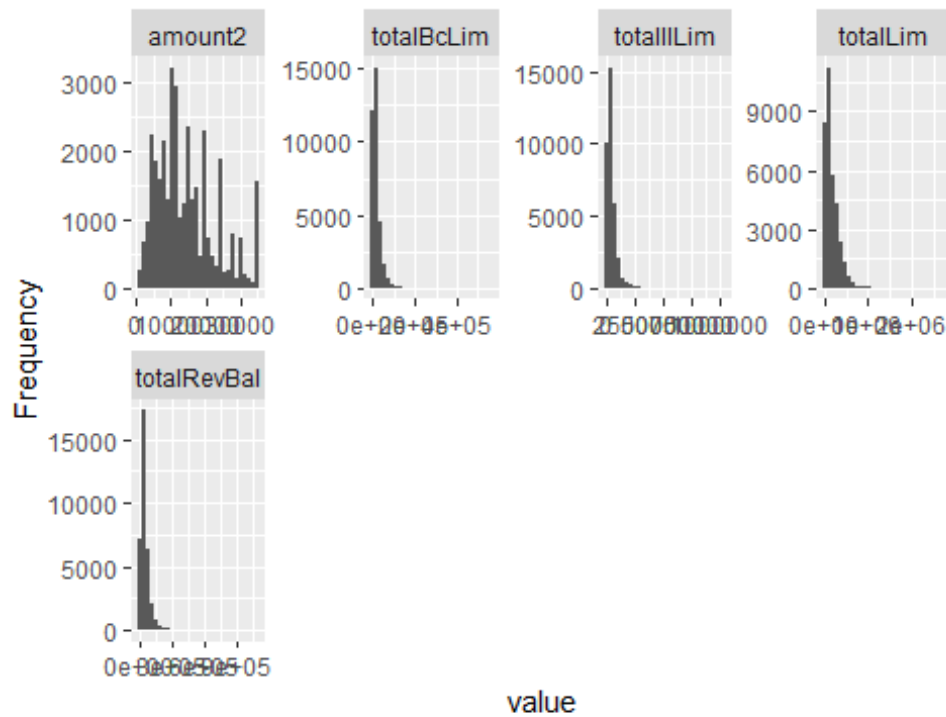


I plotted a histogram of all the variables in the LoansDT dataframe to get an overall sense of if the data is normally distributed or not. Due to space limitations, I did not include this plot. Much of the data looks right skewed.

```
plot_histogram(LoansDT)
```



Page 1

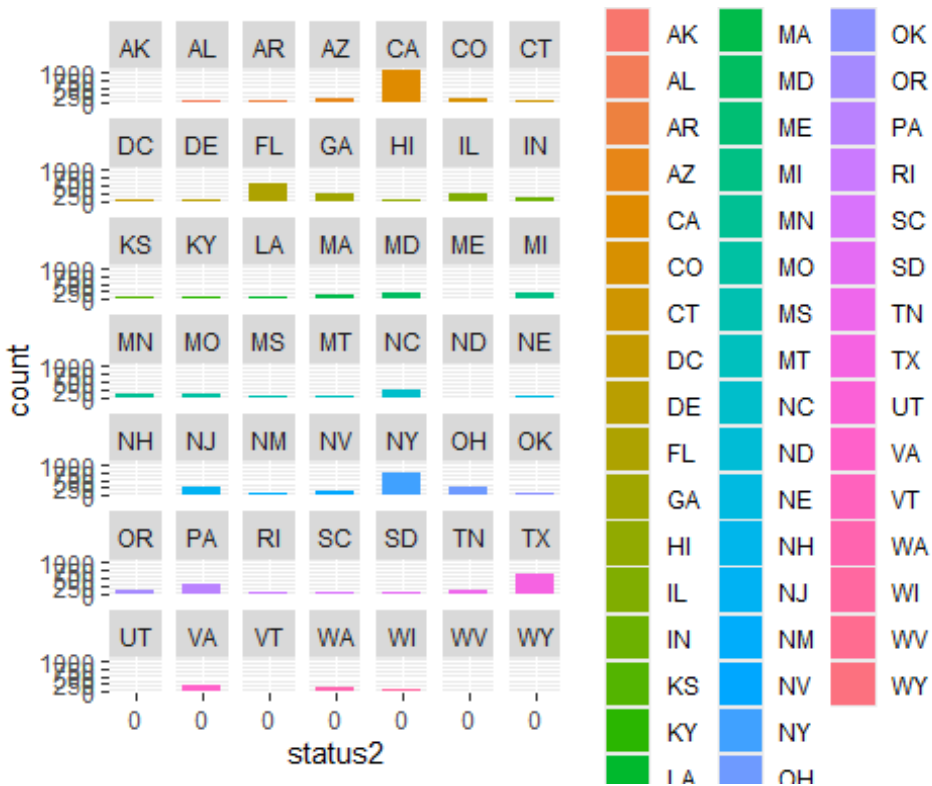


Page 2

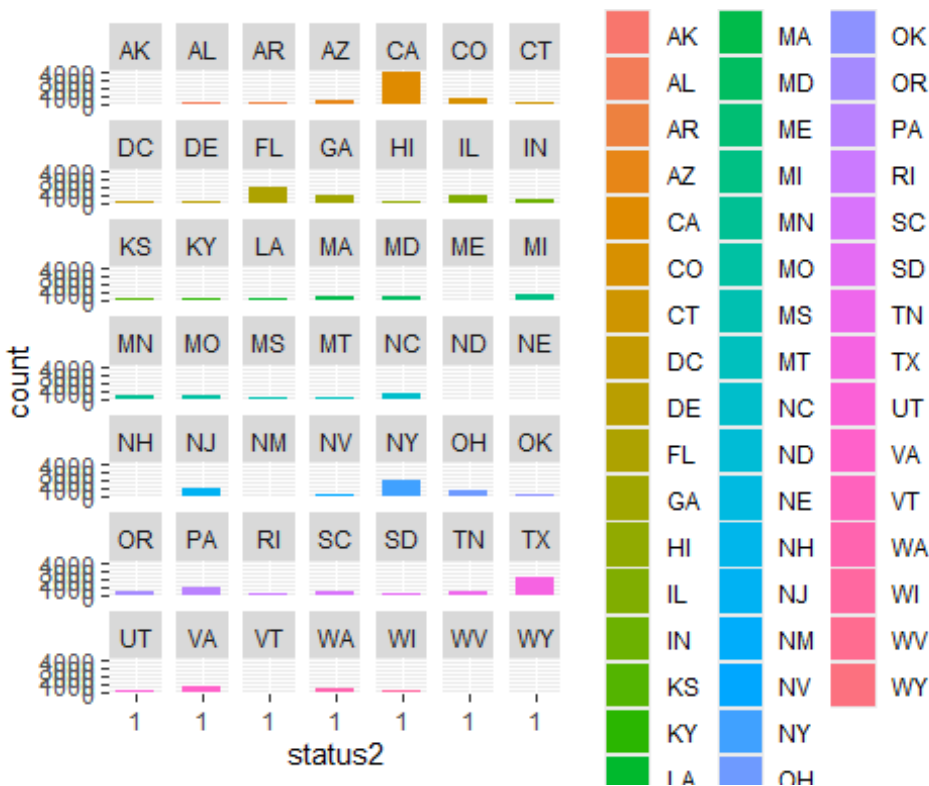
I plotted the states against status for both the good and bad loans but noticed any relationship. As expected the more populous states (New York and California) had the higher number of loans both good and bad. I will be dropping this variable. I am also going to drop the employment field

as there are so many different values in there it will slow my regression down as there are over 15,000 distinct values, which is too many for the variable to be significant.

```
p <- ggplot(loans_bad, aes(status2, fill = state)) + facet_wrap(~state)
p + geom_bar()
```



```
p <- ggplot(loans_good, aes(status2, fill = state)) + facet_wrap(~state)
p + geom_bar()
```



```

LoansDT$state <- NULL
loans_good$state <- NULL
loans_bad$state <- NULL

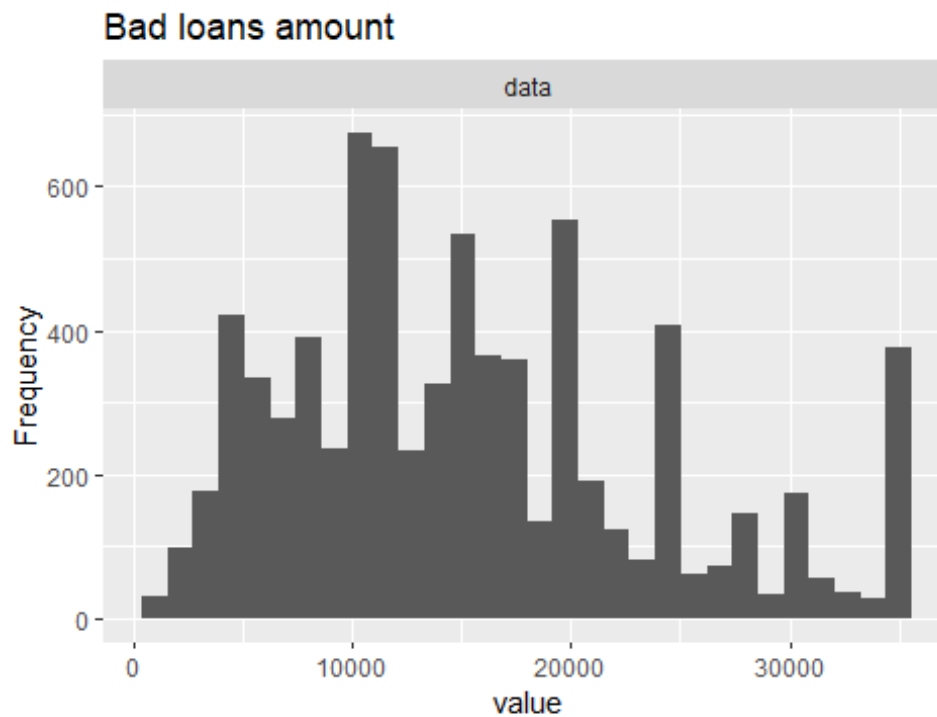
LoansDT$employment <- NULL
loans_good$employment <- NULL
loans_bad$employment <- NULL

```

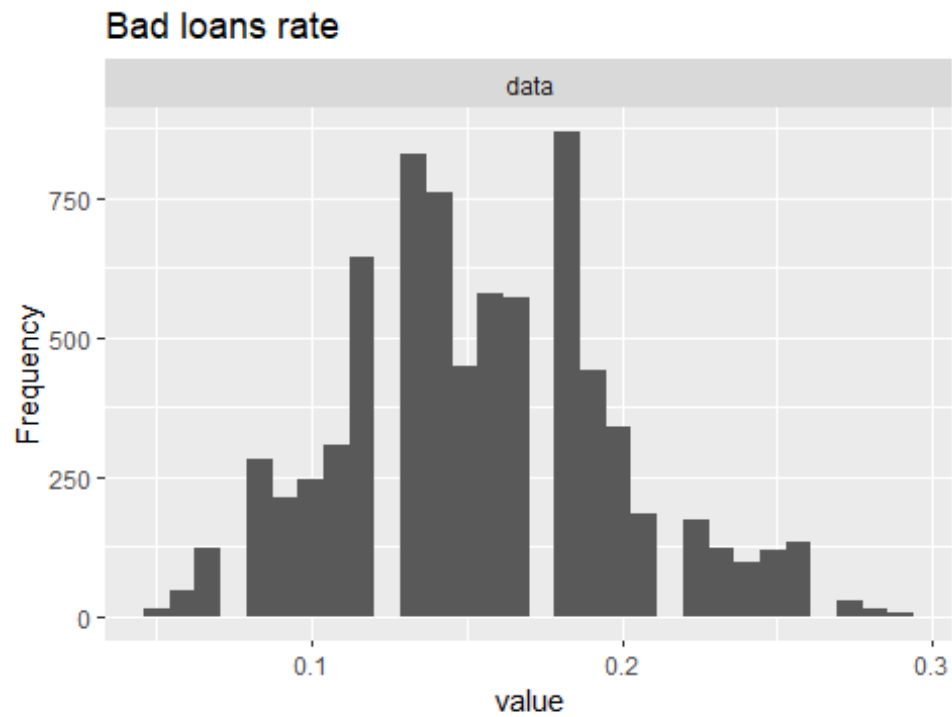
I plotted variables from each dataframe ('Good' and 'Bad') to determine if the data is normally distributed or not. There is a definite right skew to many of the numeric variables. "rate", "Amount", "debt to income ratio" look normally distributed so I will not transform them. To transform the rest of the numeric variables, I created 2 new dataframes for the good and bad loans and with just the numeric fields I am going to transform. Then I used the log1p function on the dataframes. I use this function instead of the log function because some of the variables contain zeroes and log has issues with zeroes. I will not transform the following variables as they look normal: "revolRatio", "accOpen24" and "accOpen24". I plot the data into a histogram again after the log and the data looks much more normally distributed. There are a few that now look left-skewed. Also after performing it the first time and looking for NA's I found that it created a total of 384 NAs in bcratio between the 2 groups so I am removing that from the dataframes I apply the log1p to. The debtIncRat and rate fields were also normally distributed so I did not include them in the group of variables to apply the log1p to. I also did not transform "totalPaid" as we are not supposed to use that value as a predictor. I split the categorical variables into a separate

dataframe to join with the logged dataframes later. I plotted several of the variables after the log was applied to the to check the results.

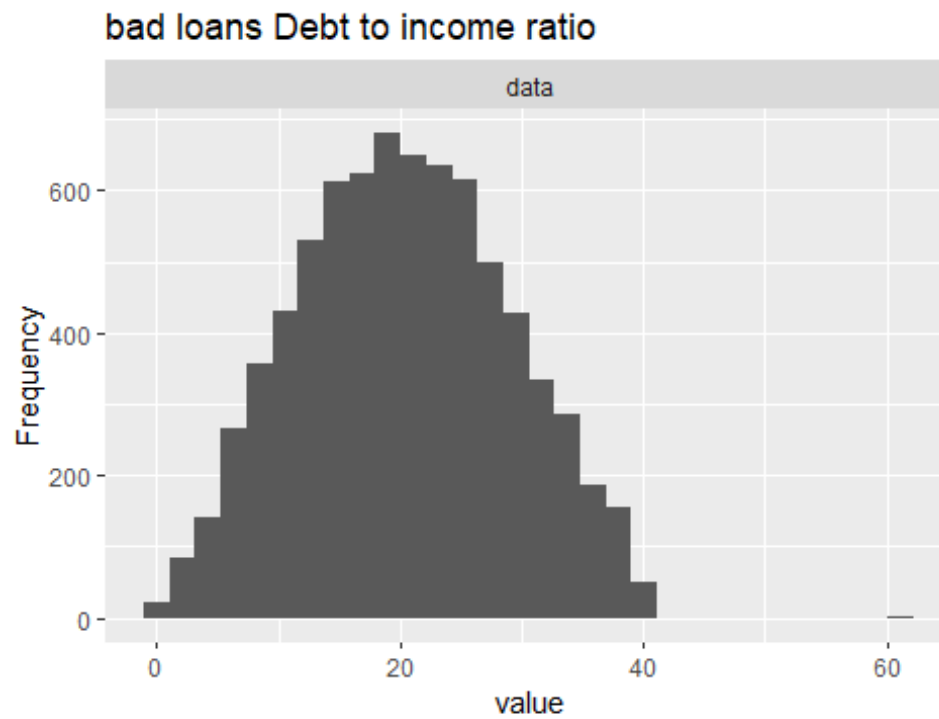
```
par(mfrow=c(4,2))  
plot_histogram(loans_bad$amount,title="Bad loans amount")
```



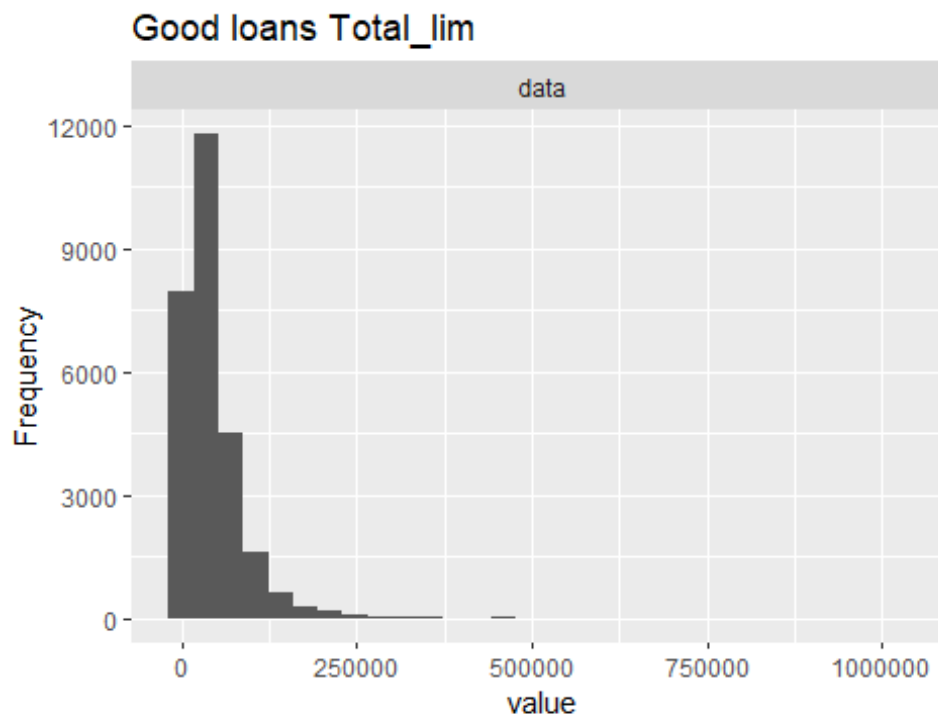
```
plot_histogram(loans_bad$rate,title="Bad loans rate")
```



```
plot_histogram(loans_bad$debtIncRat,title="bad loans Debt to income ratio")
```



```
plot_histogram(loans_good$totalIllim,title="Good loans Total_lim")
```



```
nologvars <-
subset(LoansDT,select=c(bcRatio,debtIncRat,rate,totalPaid,amount2))

good_loan_numeric <- loans_good [ c("amount" , "payment" , "income" ,
"openAcc" , "revolRatio" , "totalAcc" ,
"totalBal" , "totalRevLim" ,
"accOpen24" , "avgBal" , "bcOpen" , "totalLim",
"totalRevBal" , "totalBcLim" ,
"totalIllLim")]

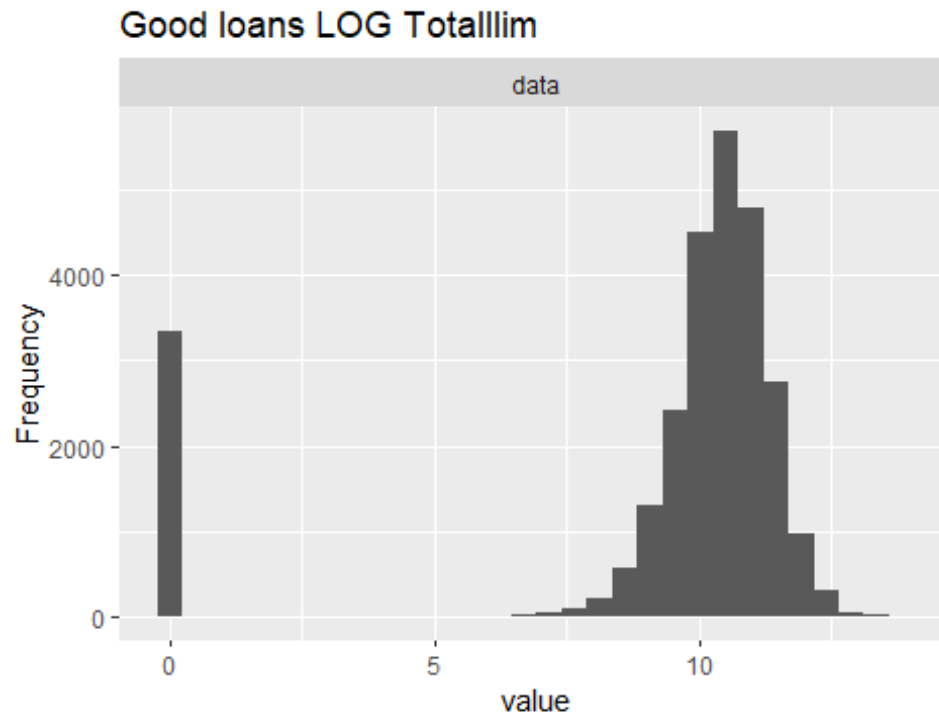
bad_loan_numeric <- loans_bad[ c("amount" , "payment" , "income" , "openAcc"
, "revolRatio" , "totalAcc" ,
"totalBal" , "totalRevLim" ,
"accOpen24" , "avgBal" , "bcOpen" , "totalLim",
"totalRevBal" , "totalBcLim" ,
"totalIllLim")]

good_loan_discrete <- loans_good[ c("grade", "length" , "home"
,"verified","reason" ,
"status2","term" )]

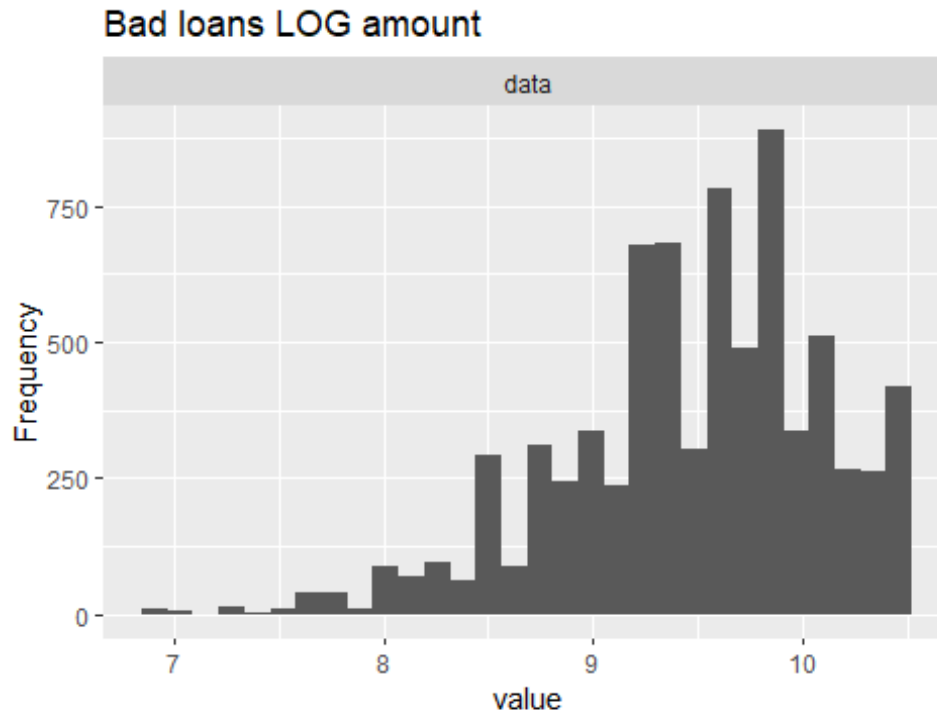
bad_loan_discrete <- loans_bad[ c("grade","length" , "home"
,"verified","reason",
"status2","term")]

good_loan_log <- log1p(good_loan_numeric)
bad_loan_log <- log1p(bad_loan_numeric)
```

```
plot_histogram(good_loan_log$totalIllim,title="Good loans LOG TotalIllim")
```



```
plot_histogram(bad_loan_log$amount,title="Bad loans LOG amount")
```



I checked the dataframes resulting from the log1p function for NA's and found none. I am not showing it to conserve space.

Next I will do some density plots with some of the different logged variables too see if there is a noticeable difference between good and bad loans. I did density plots of openacc and totalLim for good and bad loans. I didn't see much difference for openacc or totalLim between good or bad loans.

I wasn't able to eliminate many fields in my data exploration but I suspect once I start building my model, that is when I will eliminate more fields.

PART 2 - Section 5 - The Logistic Model For the start of Section 5, I will create 2 datasets from the Loan_regrsn dataframe from Step 4. One dataset will be my training dataset and will contain 80% of the data, the other will be my test dataset and will contain 20% of the data. I end up 6931 in Loan_test and 27724 in Loan_training

Next I will run the full model and use the summary function to see which variables are significant and I should include in my model. Based upon the coefficients with significant p values I will keep the following values:

grade,verified,reason,term,income,revolRatio,totalAcc,totalRevLim,accOpen24,bcOpen,t
otalRevBal,totallLim. When I run this model the p=values for the following fields are no longer significant:bcOpen,totalRevBal and totallLim, so I will drop then and try a third time. A strange thing happened When I ran the third model, T p-values was significant at .013 but it was higher than my second model which had a p-value of .012. The McFadden pseudo R Squared for the third model (.09414) was less than the second model (.09462)

but just barely. Looking at the model only 3 of the 12 values of the reason variable were significant so I decided to drop the reason variable and try it again. The results were much better for the 4th time. The p-value is .0006 with a McFadden Pseudo R2 of .0932. This is the model I will move forward with. To save space, I will only show the p-value for the first and second models.

```
Loantrain_one <- glm(status2~.,data=Loan_training,family="binomial")

r1 <- PseudoR2(Loantrain_one)

Loantrain_two <- glm(status2~grade+ verified + reason + term + income +
revolRatio +totalAcc + totalRevLim + accOpen24 + bcOpen + totalRevBal
+totalIllim ,data=Loan_training,family="binomial")

r2 <- PseudoR2(Loantrain_two)

Loantrain_three <- glm(status2~grade+ verified + reason + term + income +
revolRatio +totalAcc + totalRevLim + accOpen24
,data=Loan_training,family="binomial")

r3 <- PseudoR2(Loantrain_three)

Loantrain_Four <- glm(status2~grade + verified + term + income + revolRatio
+totalAcc + totalRevLim + accOpen24 ,data=Loan_training,family="binomial")
r4 <- PseudoR2(Loantrain_Four)

PseudoR2(Loantrain_Four, which = "all")
```

##	McFadden	McFaddenAdj	CoxSnell	Nagelkerke
AldrichNelson				
##	9.326458e-02	9.230004e-02	9.303905e-02	1.433486e-01
8.896766e-02				
##	VeallZimmermann	Efron	McKelveyZavoina	Tjur
AIC				
##	1.739347e-01	9.826169e-02	1.719503e-01	9.804239e-02
2.634995e+04				
##	BIC	logLik	logLik0	G2
##	2.646517e+04	-1.316098e+04	-1.451468e+04	2.707412e+03

Below I will use my model created above to predict the status for loans in the test data. After I use my model to predict the status of the loans, I will create a confusion matrix to

determine the overall accuracy of the model. The results of the confusion matrix for the model are as follows: I have 5260 loans correctly predicted as good loans and 154 loans correctly predicted as bad loans. So I have a total accuracy percentage of 78%. 98 percent of good loans were correctly predicted as good while only 11 percent of bad loans were correctly predicted as bad. While this model leaves a bit to be desired in predicting the bad loans accurately, it does a much better job accurately predicting the good ones.

```
pchisq(29029 - 26322, df = 13, lower.tail = FALSE)

## [1] 0

models <- list(Loantrain_one, Loantrain_two, Loantrain_three, Loantrain_Four)

comparison <- data.frame(
  Model      = c("1 - All variables", "2 - Significant variables",
                 "3 - Dropped bcOpen, totalRevBal, totalIllLim", "4 - Final
(dropped reason)"),
  Predictors = sapply(models, function(m) length(coef(m)) - 1),
  AIC        = round(sapply(models, AIC)),
  McFadden   = round(sapply(models, function(m) PseudoR2(m, "McFadden")), 3)
)
knitr::kable(comparison, caption = "Model comparison")
```

Model comparison

Model	Predictors	AIC	McFadden
1 – All variables	52	26140	0.103
2 – Significant variables	28	26340	0.095
3 – Dropped bcOpen, totalRevBal, totalIllLim	25	26348	0.094
4 – Final (dropped reason)	13	26350	0.093

```
summary(Loantrain_Four)

##
## Call:
## glm(formula = status2 ~ grade + verified + term + income + revolRatio +
##      totalAcc + totalRevLim + accOpen24, family = "binomial",
##      data = Loan_training)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -1.25375    0.36781  -3.409 0.000653 ***
## grade1       -0.49820    0.06946  -7.172 7.37e-13 ***
## grade2       -0.97831    0.06865 -14.250 < 2e-16 ***
```

```

## grade3          -1.26149    0.07319 -17.235 < 2e-16 ***
## grade4          -1.41071    0.08100 -17.417 < 2e-16 ***
## grade5          -1.66973    0.10347 -16.137 < 2e-16 ***
## grade6          -1.76051    0.17298 -10.177 < 2e-16 ***
## verifiedVerified -0.20589    0.03710  -5.550 2.86e-08 ***
## term60 months   -0.64593    0.03737 -17.286 < 2e-16 ***
## income           0.34222    0.03426   9.990 < 2e-16 ***
## revolRatio      -0.62001    0.10649  -5.822 5.81e-09 ***
## totalAcc         0.10785    0.03864   2.791 0.005248 **
## totalRevLim      0.06733    0.02164   3.112 0.001859 **
## accOpen24        -0.45874    0.03239 -14.165 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 29029  on 27723  degrees of freedom
## Residual deviance: 26322  on 27710  degrees of freedom
## AIC: 26350
##
## Number of Fisher Scoring iterations: 5

test_model <- glm(status2~grade + verified + term + income + revolRatio
+totalAcc + totalRevLim + accOpen24 ,data=Loan_training,family="binomial")
probabilities <- predict(test_model,newdata=Loan_test, type="response")

t <- .5

Bad_or_Good <- ifelse(probabilities>t,1,0)
Bad_or_Good <- as.factor(Bad_or_Good)

cf <- confusionMatrix(data=Bad_or_Good,reference=Loan_test$status2)
cf$table

##           Reference
## Prediction    0    1
##           0  126  106
##           1 1429  5270

cf$overall[1]

## Accuracy
## 0.7785312

```

Section 6 - Optimizing the Threshold for Accuracy.

In this section I am going to vary the threshold from .5 to attempt to correctly predict more bad loans. I will create a confusion matrix for each different threshold and then I will graph them all to show accuracy vs. threshold. As you recall from the previous section my

accuracy percentage was 78% but was a much better predictor of 'good' loans than 'bad' loans. I will calculate the accuracy for 5 different thresholds between 0 and 1 to attempt to predict a better percentage of bad loans. Due to space constraints, I will just show the confusion matrix containing the predictions and the accuracy value for each calculation.

```
# 6 different thresholds
```

```
th1 <- .30  
th2 <- .45  
th3 <- .55  
th4 <- .60  
th5 <- .75
```

```
Bad_or_Good1 <- ifelse(probabilities>th1,1,0)  
Bad_or_Good1 <- as.factor(Bad_or_Good1)  
c1 <- confusionMatrix(data=Bad_or_Good1,reference=Loan_test$status2)  
c1$table
```

```
##           Reference  
## Prediction    0    1  
##           0    0    0  
##           1 1555 5376
```

```
c1$overall[1]
```

```
## Accuracy  
## 0.7756456
```

```
Bad_or_Good2 <- ifelse(probabilities>th2,1,0)  
Bad_or_Good2 <- as.factor(Bad_or_Good2)  
c2 <- confusionMatrix(data=Bad_or_Good2,reference=Loan_test$status2)  
c2$table
```

```
##           Reference  
## Prediction    0    1  
##           0   55   35  
##           1 1500 5341
```

```
c2$overall[1]
```

```
## Accuracy  
## 0.7785312
```

```
Bad_or_Good3 <- ifelse(probabilities>th3,1,0)  
Bad_or_Good3 <- as.factor(Bad_or_Good3)  
c3 <- confusionMatrix(data=Bad_or_Good3,reference=Loan_test$status2)  
c3$table
```

```
##           Reference  
## Prediction    0    1
```

```
##           0   215   204
##           1 1340  5172

c3$overall[1]

## Accuracy
## 0.7772327

Bad_or_Good4 <- ifelse(probabilities>th4,1,0)
Bad_or_Good4 <- as.factor(Bad_or_Good4)
c4 <- confusionMatrix(data=Bad_or_Good4,reference=Loan_test$status2)
c4$table

##           Reference
## Prediction    0    1
##           0   349   381
##           1 1206  4995

c4$overall[1]

## Accuracy
## 0.7710287

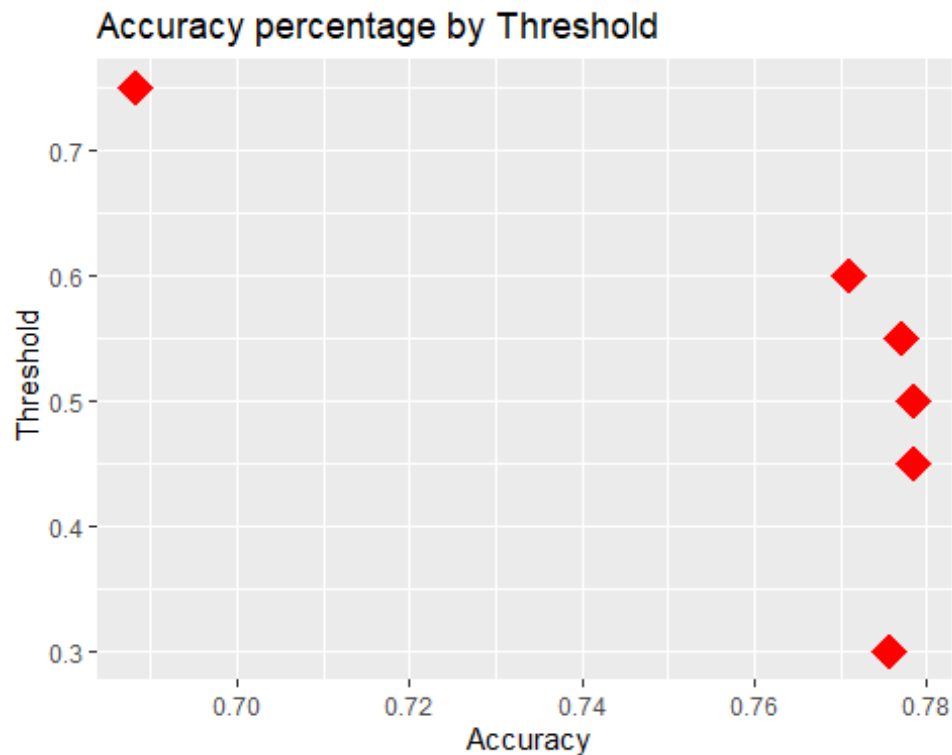
Bad_or_Good5 <- ifelse(probabilities>th5,1,0)
Bad_or_Good5 <- as.factor(Bad_or_Good5)
c5 <- confusionMatrix(data=Bad_or_Good5,reference=Loan_test$status2)
c5$table

##           Reference
## Prediction    0    1
##           0   908 1513
##           1   647 3863

c5$overall[1]

## Accuracy
## 0.6883567
```

I compared accuracy across six thresholds: 0.30, 0.45, 0.50 (default), 0.55, 0.60, and 0.75. Accuracy peaks at 77.9% at both 0.45 and 0.50, but that figure is misleading: simply predicting every loan as good (the 0.30 threshold) scores 77.6% while catching none of the 1,555 bad loans. Raising the threshold trades accuracy for catching more bad loans. At the default of 0.50, the model correctly identifies 126 bad loans; at 0.60, 349; and at 0.75, 908 (58% of all bad loans), though overall accuracy falls to 68.8% because 1,513 good loans are also flagged as bad. Because accuracy doesn't reflect the different costs of these errors, Part 7 evaluates thresholds by profit instead.



Part 7 - Optimizing

the Threshold for Profit In this section I will take the predictions at the different threshold levels from Part 6 and apply them to my test data. Then I will sum the profit for each threshold and determine what the maximum profit increase is for the loans I predicted as good. I add the probabilities calculations from section 5 and the test data I created in section 5. Then I sum the total paid minus the amount for my total profit. According to the model, the lowest threshold of .3 is where the biggest profit is - 1,418,710. For there the profit for the .45 threshold is 1,374,321. The default threshold profit (.5) is 1,386,706. The profit for .55 is 1,354,828. The profit keeps decreasing from there. At .65 the profit is 98,274.20 then for .75 threshold the profit is 966,017.50. The total profit for all the loans in the training set is 1,418,710. The profit for a 'perfect' model that denies all of the truly bad loans is 750778.40.

```
Loan_test_th1 <- cbind(Loan_test,probabilities)

thr1 <- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1 where
probabilities >= .30"))

thr2<- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1 where
probabilities >= .45 "))

thrdf <- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1 where
probabilities >= .50 "))

thr3<- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1 where
```

```

probabilities >= .55 "))

thr4<- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1 where
probabilities >= .65 "))

thr5<- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1 where
probabilities >= .75 "))

Prof_no_model <- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1
where status2 = 1 "))

tot_profit <- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1"))

thr1

## sum(totalPaid - amount2)
## 1 1418710

thr2

## sum(totalPaid - amount2)
## 1 1374321

thrdf

## sum(totalPaid - amount2)
## 1 1386706

thr3

## sum(totalPaid - amount2)
## 1 1354828

thr4

## sum(totalPaid - amount2)
## 1 982747.2

thr5

## sum(totalPaid - amount2)
## 1 966017.5

Prof_no_model

## sum(totalPaid - amount2)
## 1 750778.4

tot_profit

## sum(totalPaid - amount2)
## 1 1418710

```

```

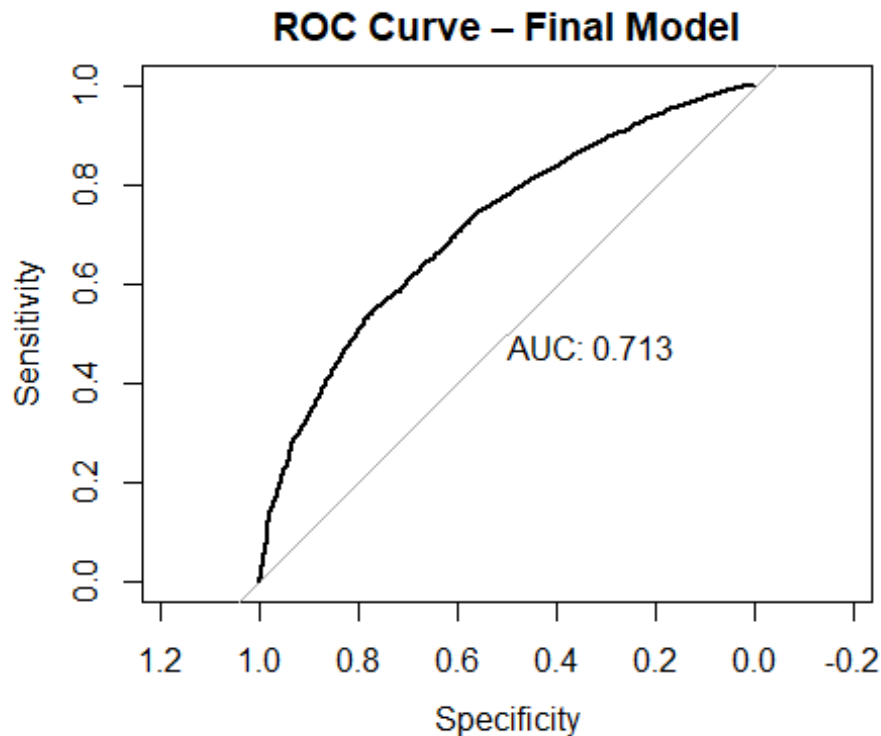
# Predicted probabilities on the test set
library(pROC)

probs <- predict(Loantrain_Four, newdata = Loan_test, type = "response")
roc_obj <- roc(Loan_test$status2, probs)
auc(roc_obj)

## Area under the curve: 0.7128

plot(roc_obj, print.auc = TRUE, main = "ROC Curve - Final Model")

```



```

all_profit <- sum(Loan_test_th1$totalPaid - Loan_test_th1$amount2)

thresholds <- seq(0.30, 0.90, by = 0.01)
profit <- sapply(thresholds, function(t)
  with(Loan_test_th1, sum((totalPaid - amount2)[probabilities >= t])))

best_t <- thresholds[which.max(profit)]
all_profit      # approve everyone

## [1] 1418710

max(profit)     # best threshold's profit

## [1] 1438801

best_t          # the best threshold

```

```
## [1] 0.38
```

```
plot(thresholds, profit, type = "l",  
     xlab = "Approval threshold (P(good))", ylab = "Profit",  
     main = "Test-Set Profit by Threshold")  
abline(h = all_profit, col = "red", lty = 2)
```

